

Лабораторная работа №1

Вычисление числа π с использованием OpenMP

Выполнил: Москаленко Дмитрий, группа 11-401

1. Описание работы

Реализованы последовательная и параллельная версии вычисления числа π по ряду Лейбница на языке C. Параллельная версия использует директиву `#pragma omp parallel for reduction(+:sum)`, время выполнения измерялось функцией `omp_get_wtime()`. Эксперименты проводились на процессоре с 8 ядрами (12 логических процессоров).

2. Результаты экспериментов

Таблица 1. Влияние количества потоков ($n = 10^9$)

Потоки	Время, с	Ускорение
1	1,915	1,01
2	0,983	1,96
4	0,512	3,72
8	0,344	5,55
16	0,278	6,88

Таблица 2. Влияние размера задачи (8 потоков)

n	Последовательно, с	OpenMP (8 потоков), с	Ускорение
10^7	0,020	0,006	3,33
10^8	0,189	0,040	4,73
10^9	1,906	0,342	5,57
10^{10}	19,105	3,435	5,56

3. Вывод

При увеличении числа потоков время выполнения уменьшалось: с 1,915 с на одном потоке до 0,278 с на 16 потоках. До 4 потоков ускорение было близко к линейному (3,72 на 4 потоках). Начиная с 8 потоков рост ускорения заметно замедлился: 5,55 на 8 потоках и 6,88 на 16 потоках. Это объясняется архитектурой процессора: 8 физических ядер и 12 логических процессоров за счёт SMT, который не удваивает производительность ядра. Потоки сверх числа логических процессоров ($16 > 12$) почти не дают выигрыша, так как лишние потоки ожидают своей очереди на выполнение. В эксперименте с фиксированным числом потоков при малом размере задачи ($n = 10^7$) ускорение составило лишь 3,33, так как накладные расходы на создание потоков и объединение результатов сопоставимы со временем вычислений. С ростом n ускорение увеличивается и при $n \geq 10^9$ выходит на предел около 5,6 для 8 потоков. Таким образом, OpenMP эффективен для больших задач, а максимальное ускорение ограничено числом ядер процессора.